

LLM-YARA: A Structured LLM Pipeline for Automated YARA Rule Generation

Final Technical Report

Yassir BOUSSETA

March 2026

Abstract

We present LLM-YARA, a static-analysis-only pipeline that uses a large language model (LLM) with structured generation and deterministic rendering to produce YARA rules for malware family detection. Evaluated on 138 families from the MOTIF dataset under identical train/test splits, LLM-YARA achieves a mean F_1 of **0.4240** with a mean benign false-positive rate (FPR) of **0.0%** and mean rule specificity of **0.9988**. This represents a non-significant F_1 difference of +0.0044 ($p = 0.85$) relative to the TOPSTRINGS baseline (mean $F_1 = 0.4195$), which itself achieves higher recall (0.4797 vs. 0.4043) but lower specificity (0.9924 vs. 0.9988). The pipeline rejects 10 of 138 families through acceptance gates, yielding 128 accepted rules. We report all-family metrics as headline numbers and provide ok-only figures as a secondary view. Results are reported from saved artifacts and checked once with replay.

1 Introduction

YARA rules are a cornerstone of malware detection in operational security workflows [6]. Analysts write rules that match byte patterns, string literals, imports, and structural properties of Portable Executable (PE) files to classify samples into malware families. Manual authoring of these rules does not scale: new families emerge continuously, and maintaining high specificity across large benign corpora is labor-intensive.

Automated YARA generation approaches have explored deterministic string selection [3], static feature heuristics [8], dynamic-trace synthesis [7], and packer-aware pipelines [1]. Recent work has also investigated LLM-guided rule generation for intrusion detection signatures [9].

This report describes LLM-YARA, a pipeline that occupies a specific point in the design space: *static-only feature extraction, structured primary LLM payloads, deterministic rendering* of YARA syntax, and *acceptance gates* that reject rules failing minimum recall or maximum FPR thresholds on held-out development data.

Study question. Can a static-only LLM pipeline produce YARA rules on the same split as deterministic baselines while keeping false positives low?

We do not claim that LLM-YARA beats TOPSTRINGS in F_1 : the difference is tiny and not statistically significant. In this evaluation, LLM-YARA reaches mean $F_1 = 0.4240$, mean benign FPR= 0.0000, and 128 accepted families out of 138.

2 Dataset and Split Discipline

2.1 The MOTIF Dataset

We use the MOTIF malware reference dataset [2], which provides ground-truth family labels for PE malware samples. After filtering for families with at least 6 samples (`min_family_size=6`), the evaluation covers **138 malware families**.

2.2 Split Design

All splits are deterministic (`seed=42`) and computed per-family:

- **train_target / test_target:** 70%/30% of each family’s samples (`test_ratio=0.3`).
- **benign_dev / benign_test:** The benign corpus is split 60%/40% (`benign_dev_ratio=0.6`).

Leakage prevention. LLM prompting uses `train_target` samples from the family under construction. Feature selection ranks each family’s `train_target` positives against `train_other` plus `benign_dev` negatives. Final metrics are computed exclusively on `test_target` and `benign_test`, which remain unseen during generation and final scoring. No baseline receives information from the test split.

3 System Design and Implementation

LLM-YARA consists of six sequential stages, illustrated in Figure 1.

3.1 Stage 1: Index

A manifest is built for every sample, recording its SHA-256 hash, family label, data source, and file-system path. This manifest anchors all downstream references.

3.2 Stage 2: Split

Deterministic per-family stratified splitting produces `train_target` (70%) and `test_target` (30%). The benign pool is split into `benign_dev` (60%) and `benign_test` (40%). All splits are seeded (`seed=42`) for reproducibility.

3.3 Stage 3: Extract

Static PE features are extracted for every indexed sample: printable ASCII and wide strings, PE import names (DLL and function), section names, and PE metadata (timestamps, entry-point characteristics). No dynamic or behavioral features are used.

3.4 Stage 4: Select

Discriminative features are ranked with TF-IDF vectorization [5] and chi-squared scoring. For each family, the selector treats `train_target` as positives and `train_other` plus `benign_dev` as negatives. The top $k = 30$ features per family are retained (`top_k_features=30`).

3.5 Stage 5: Generate

The curated feature set is formatted into a structured prompt and sent to the configured LLM backend (gpt-5-mini). The model returns a **JSON payload** specifying:

- Selected string literals (up to `max_strings=20`),
- Import predicates (DLL + function pairs),
- Section-based constraints,
- A condition skeleton (e.g., threshold count of string matches).

A **deterministic renderer** converts this JSON into syntactically valid YARA source. The primary generation path stays structured, while repair payloads may preserve full YARA text returned by the model.

Compile and repair. The rendered rule is compiled with `yara-python`. If compilation fails, the error mes-

sage is fed back to the LLM for a repair attempt, up to `max_repairs=3`.

Train-hit gate. The compiled rule is scanned against `train_target`. If fewer than `min_train_target_hits=1` sample matches, the family is rejected.

Benign-dev FPR gate. The rule is scanned against `benign_dev`. If the FPR exceeds `benign_dev_fpr_threshold=0.02`, the family is rejected.

3.6 Stage 6: Evaluate

Accepted rules are scanned against `test_target` and `benign_test`. Per-family metrics include: true positive rate (TPR), false positive rate (FPR), F_1 score, rule specificity ($1 - \text{off-target rate}$), and rule complexity (string count plus condition clause count).

3.7 Implementation and Packaging

The repository ships as a complete CLI workflow with `Dockerfile`, `docker-compose.yml`, and `start.sh`. The default entrypoint validates the requested mode and required dependencies before dispatching either a demo run or the full `index→split→extract→select→generate→evaluate` pipeline. Saved outputs under `artifacts/final_motif_audit/` provide the numbers reported in this document, while replay support allows a second pass over cached LLM responses without changing the evaluation contract.

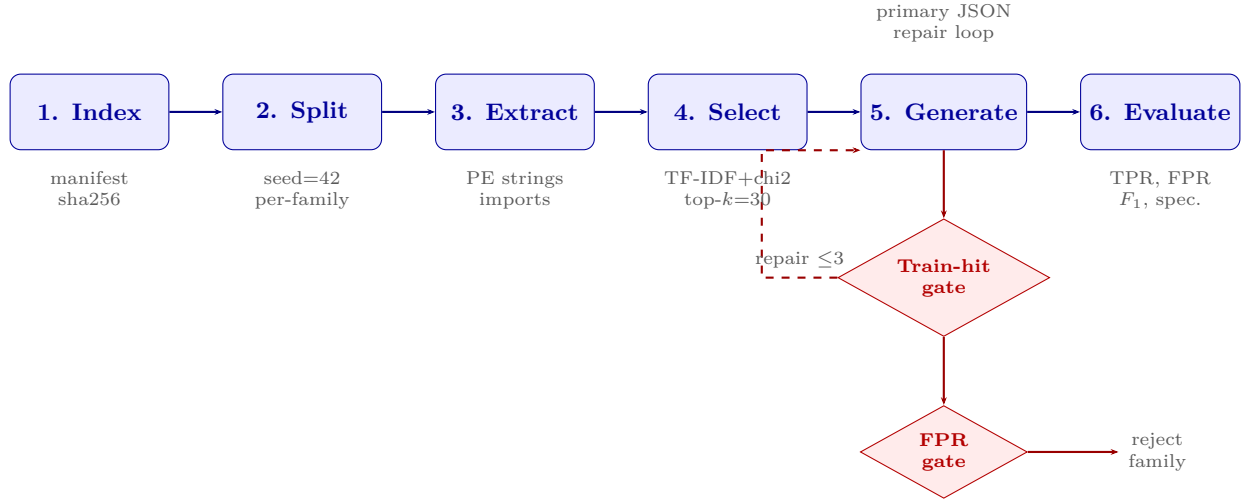


Figure 1: LLM-YARA pipeline. Stages 1–4 are fully deterministic. Stage 5 queries the LLM for a structured JSON payload, renders YARA deterministically, and applies two acceptance gates. Stage 6 evaluates on the held-out test split.

4 Experimental Protocol

4.1 Headline vs. Secondary Metrics

All-family metrics (138 families, failed families scored as zero) are the headline results. This penalizes LLM-YARA for its 10 rejected families and prevents inflated numbers. **Ok-only metrics** (128 accepted families) are reported separately in Appendix A as a secondary view of rule quality among accepted families.

4.2 Significance Testing

Pairwise comparisons use a **paired randomization test** with 5,000 iterations and seed=42 [4]. For each pair of methods, per-family F_1 differences are randomly permuted to build a null distribution. A two-sided p -value below 0.05 is required for significance.

4.3 Configuration Summary

Table 1 lists all pipeline parameters.

Table 1: Pipeline configuration parameters.

Parameter	Value
test_ratio	0.3
benign_dev_ratio	0.6
min_family_size	6
top_k_features	30
max_strings	20
max_repairs	3
benign_dev_fpr_threshold	0.02
min_train_target_hits	1
temperature_config	1.0
seed	42
model	gpt-5-mini
backend	openai

5 Executed Methods

Only methods executed within this repository appear in the main comparison table. Table 2 lists each method and its status.

Table 2: Methods in the executed comparison set.

Method	Status	Note
LLM-YARA	Primary	This work
TOPSTRINGS	Reproduced	Deterministic string baseline
APIARY-STATIC	Re-implemented	Static proxy of [8]
AUTOYARA-BICLUSTER	Re-implemented	Byte- n -gram biclustering

YAMME [7] and PACKGENOME [1] are discussed in Section 9 as related work only. Neither was executed in this evaluation, and no numbers from those papers ap-

pear in our comparison tables. RULELLM [9] informed the structured-payload design but targets intrusion-detection signatures rather than YARA rules.

6 Main Results

Table 3 presents all-family metrics across the four executed methods.

LLM-YARA has the numerically highest mean F_1 (0.4240) and the highest mean specificity (0.9988) with zero mean benign FPR. However, the F_1 difference from TOPSTRINGS is marginal: only +0.0044 absolute.

Source: artifacts/final_motif_audit/comparison_summary.json.

TOPSTRINGS delivers higher mean TPR (0.4797 vs. 0.4043) and covers all 138 families. Its specificity (0.9924) is lower than LLM-YARA but still high in absolute terms.

APIARY-STATIC and AUTOYARA-BICLUSTER achieve substantially higher recall (0.6999 and 0.7965 respectively) but at the cost of high benign FPR (0.0718 and 0.1183) and much lower specificity (0.8638 and 0.8180), resulting in lower F_1 scores.

Mean rule complexity for LLM-YARA (11.78) is higher than TOPSTRINGS (8.94) and APIARY-STATIC (8.51), reflecting the multi-clause conditions produced by the structured renderer (mean condition clause count of 2.21 vs. 1.0 for both baselines). Mean string count is 9.57 for LLM-YARA vs. 7.94 for TOPSTRINGS.

Example accepted rule. The Nanolocker family reaches $F_1 = 1.0$, TPR= 1.0, and benign FPR= 0.0. The saved rule combines a PE header check, an import predicate, and thresholded string matching:

```
strings:
  $a1 = "inet_addr" ascii
condition:
  uint16(0) == 0x5A4D and
  pe.imports("advapi32.dll",
    "abortsystemshutdownna") and
  1 of them
```

This snippet shows the final rule shape used by the primary method. The full annotated rule remains in

Table 3: Main comparison (all-family metrics, 138 families). Rejected families are scored as $F_1 = 0$, TPR = 0. Best values per metric in **bold**; second-best underlined.

Method	OK	F_1	TPR	FPR _{ben}	Spec.	Comp.
LLM-YARA	128 / 138	0.4240	0.4043	0.0000	0.9988	11.78
TOPSTRINGS	138 / 138	0.4195	0.4797	0.0061	0.9924	8.94
APIARY-STATIC	138 / 138	0.1557	0.6999	0.0718	0.8638	8.51
AUTOYARA-BICLUSTER	138 / 138	0.2760	0.7965	0.1183	0.8180	11.49

Appendix D. Source: outputs/final_motif_openai/rules/nanolocker.yar.

7 Significance & Trade-offs

7.1 Statistical Significance

Table 4 summarizes paired randomization test results.

Table 4: Paired randomization tests (5,000 iterations, seed=42) comparing LLM-YARA against each baseline on per-family F_1 .

Comparison	ΔF_1	p -value	Sig.?
vs. TOPSTRINGS	+0.0044	0.8484	No
vs. APIARY-STATIC	+0.2683	0.0002	Yes
vs. AUTOYARA-BICLUSTER	+0.1480	0.0002	Yes

The comparison with TOPSTRINGS is **not significant** ($p = 0.85$). The two methods are statistically tied on F_1 . The differences against APIARY-STATIC and AUTOYARA-BICLUSTER are highly significant ($p < 0.001$).

Source: artifacts/final_motif_audit/significance_vs_primary.json.

7.2 Trade-off Structure

The four methods occupy distinct positions in the recall–specificity trade-off space, visualized in Figure 2.

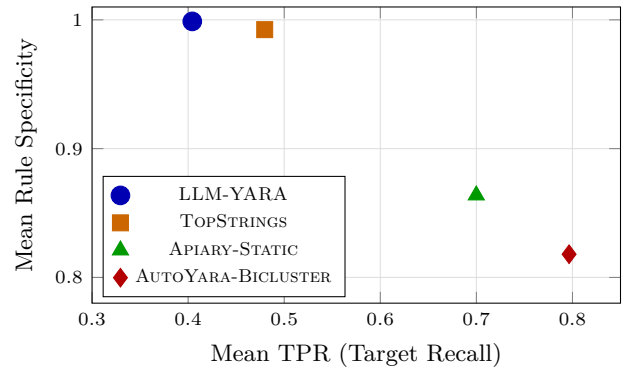


Figure 2: Recall–specificity trade-off. LLM-YARA occupies the high-specificity / lower-recall quadrant. AUTOYARA-BICLUSTER and APIARY-STATIC achieve higher recall at substantially lower specificity.

LLM-YARA and TOPSTRINGS both occupy the high-specificity region, with LLM-YARA slightly higher on specificity and TOPSTRINGS slightly higher on recall. APIARY-STATIC and AUTOYARA-BICLUSTER trade specificity for recall, which reduces their F_1 due to high false-positive rates.

The clearest difference for LLM-YARA is specificity: 0.9988 vs. 0.9924 for TOPSTRINGS. On this split, the lower benign FPR is the main reason why LLM-YARA remains numerically first on mean F_1 despite lower recall.

8 Failure Analysis

Of 138 families, LLM-YARA rejects 10 through its acceptance gates. Table 5 categorizes these rejections.

Table 5: Rejection reasons for the 10 failed families.

Rejection Reason	Count
train_target_hits_below_min	7
benign_dev_fpr_exceeded	3
Total rejected	10

Train-hit failures (7 families): Carbanak, Dridex, MosaicRegressor, Nazar, Nymaim, Phorpiex, SmokeLoader. These families have samples where the LLM-selected features do not produce rules matching even one training sample after up to 3 repair attempts. Possible causes include packed or obfuscated samples where static strings are absent, or families whose discriminative features are behavioral rather than static.

Benign-dev FPR failures (3 families): Cerber, Dharma, Ryuk. The generated rules match too many benign samples in `benign_dev`, exceeding the 0.02 FPR threshold. This suggests the LLM-selected features for these families overlap with legitimate software patterns.

The 128/138 coverage gap (92.8% family acceptance rate) is a limitation. All three baselines produce rules for all 138 families because they lack acceptance gates. However, those rules may have high FPR (e.g., AUTOYARA-BICLUSTER achieves 0.1183 mean benign FPR across all families). The gate-based approach trades coverage for rule quality assurance.

Source: `artifacts/final_motif_audit/failure_analysis.json`.

9 Related Work

9.1 Executed Baselines

TopStrings. A deterministic baseline that selects the most frequent family-discriminative strings and compiles them into a YARA rule with a threshold condition. Reproduced from scratch in our repository and run on identical splits.

Apiary-Static [8] is treated here as **literature-inspired static related work**. We **re-implemented a static proxy** (string/import discrimination) to enable a controlled same-split comparison inside this repository.

AutoYara-Bicluster [3] uses biclustering on byte n -grams to identify common byte patterns across family members. We **re-implemented** the biclustering approach and evaluated it on our splits.

9.2 Cited-Only Methods

YAMME [7] is cited as related work on automated YARA generation. It was not executed on this repository split, and no numbers from YAMME appear in our comparison tables.

PackGenome [1] targets packed malware by generating YARA rules from unpacking traces. It addresses a complementary problem (packer detection vs. family classification) and was not executed in our evaluation.

RuleLLM [9] uses LLMs to generate Suricata and Snort rules for network intrusion detection. Its structured-payload approach (LLM outputs a constrained schema rather than freeform text) informed the design of our generation stage. RULELLM targets network signatures, not YARA rules, and was not benchmarked here.

10 Limitations and Conclusion

10.1 Limitations

Static-only scope. The pipeline extracts no dynamic, behavioral, or network features. Families whose distinguishing characteristics are runtime behaviors (API call sequences, C2 communication patterns) may be poorly served. The 7 train-hit failures likely reflect this limitation.

Tied with TopStrings on F_1 . The +0.0044 F_1 difference is not statistically significant ($p = 0.85$). TOPSTRINGS achieves higher recall (0.4797 vs. 0.4043) and covers all 138 families. This favors TOPSTRINGS if recall and full coverage matter more than benign FPR.

Incomplete family coverage. Acceptance gates reject 10 of 138 families (7.2%). While this is a deliberate quality-control mechanism, it means LLM-YARA cannot produce rules for all families in the dataset.

LLM dependency. The pipeline requires API access to a capable LLM. Cost, latency, and model availability are practical concerns. Replay stays close to the primary run but is not bit-for-bit identical, so exact equality should not be claimed.

Single dataset. All results are on MOTIF. Generalizability to other malware corpora, label sets, or PE distributions is untested.

10.2 Conclusion

LLM-YARA implements a structured LLM pipeline in which the model selects features through a constrained JSON schema and a deterministic renderer produces YARA syntax.

On MOTIF, the method reaches mean $F_1 = 0.4240$, mean benign FPR= 0.0000, mean specificity = 0.9988, and accepted rules for 128 of 138 families.

The main observations are:

- Numerically highest mean F_1 in the executed comparison, but not a significant difference from TOP-STRINGS.
- Zero benign FPR and the highest specificity among the four executed methods.
- Lower recall than APIARY-STATIC and AUTOYARA-BICLUSTER, and 10 rejected families due to acceptance gates.

This method favors low false positives over recall and full family coverage.

A Ok-Only Metrics

Table 6 reports metrics computed only over the 128 families for which LLM-YARA produced accepted rules. These numbers are **not** the headline results; they characterize rule quality among accepted families.

Table 6: Ok-only metrics (128 accepted families for LLM-YARA; all 138 for baselines, whose ok-only equals all-family).

Method	Mean F_1	Mean TPR	Mean FPR_{ben}	Mean Spec.
LLM-YARA	0.4571	0.4359	0.0000	0.9987
TOPSTRINGS	0.4195	0.4797	0.0061	0.9924
APIARY-STATIC	0.1557	0.6999	0.0718	0.8638
AUTOYARA-BICLUSTER	0.2760	0.7965	0.1183	0.8180

Among accepted families, LLM-YARA mean F_1 rises from 0.4240 to 0.4571. This reflects the zero-score penalty imposed on the 10 rejected families in the all-family view. Baselines have identical ok-only and all-family numbers because they produce rules for all families.

B Significance Test Details

The paired randomization test proceeds as follows.

Step 1. For each of the $N = 138$ families, compute the per-family F_1 difference against a baseline:

$$d_i = F_{1,i}^{(\text{LLM-YARA})} - F_{1,i}^{(\text{baseline})}.$$

Step 2. Compute the observed mean difference:

$$\bar{d} = \frac{1}{N} \sum_i d_i.$$

Step 3. For $B = 5,000$ iterations (seed=42), randomly flip the sign of each d_i with probability 0.5 and recompute a permuted mean difference $\bar{d}^*$.

Step 4. Estimate the two-sided p -value as:

$$p = \frac{|\{b : |\bar{d}_b^*| \geq |\bar{d}|\}|}{B}.$$

Results:

- **vs. TopStrings:** $\bar{d} = +0.004442$, $p = 0.84843$. Not significant. The 138-family F_1 distributions of LLM-YARA and TOPSTRINGS are statistically indistinguishable.
- **vs. Apiary-Static:** $\bar{d} = +0.268289$, $p = 0.0002$. Significant ($p < 0.001$).
- **vs. AutoYara-Bicluster:** $\bar{d} = +0.147952$, $p = 0.0002$. Significant ($p < 0.001$).

C Rejected Family List

Table 7 lists all 10 families rejected by LLM-YARA acceptance gates.

The train-hit failures cluster around families known for heavy packing, polymorphism, or small static footprints. The FPR failures involve ransomware families (Cerber, Dharma, Ryuk) whose PE characteristics may overlap with legitimate software (e.g., crypto library imports, generic string patterns).

Table 7: Rejected families and rejection reasons.

Family	Rejection Reason
Carbanak	train_target_hits_below_min
Dridex	train_target_hits_below_min
MosaicRegressor	train_target_hits_below_min
Nazar	train_target_hits_below_min
Nymaim	train_target_hits_below_min
Phorpiex	train_target_hits_below_min
SmokeLoader	train_target_hits_below_min
Cerber	benign_dev_fpr_exceeded
Dharma	benign_dev_fpr_exceeded
Ryuk	benign_dev_fpr_exceeded

D Annotated YARA Rule Example

Listing 1 shows an excerpt of the LLM-YARA-generated rule for the Nanolocker family, which achieved perfect scores ($F_1 = 1.0$, TPR = 1.0, FPR = 0.0, specificity=1.0, complexity=23). The full rule text is taken from the saved primary output at `outputs/final_motif_openai/rules/nanolocker.yar`.

```

1 import "pe"
2 rule nanolocker_candidate_01 {
3     meta:
4         author = "automated-threat-detection"
5         description = "High-signal substring candidates for Nanolocker"
6         family = "nanolocker"
7     strings:
8         $a1 = "0a-%m]mm" ascii
9         $a2 = "9j'mmlm" ascii
10        $a3 = "mmm%]mm" ascii
11        $a4 = "mmmmmmmmmmmmmmmmh)m" ascii
12        $a5 = "inet_addr" ascii
13        $a6 = "2.83+?! " ascii
14        $a7 = "\\mhavgco&" ascii
15        $a8 = "cnr|*d\\1" ascii
16        /* 12 additional strings omitted for readability */
17    condition:
18        uint16(0) == 0x5A4D and
19        pe.imports("advapi32.dll", "abortsystemshutdowna") and
20        1 of them
21 }
```

Listing 1: Excerpt of the Nanolocker rule generated by LLM-YARA.

Annotation. The rule combines three condition layers:

1. **PE header check:** `uint16(0) == 0x5A4D` confirms the file is a PE binary.
2. **Import predicate:** `pe.imports("advapi32.dll", "abortsystemshutdowna")` targets a distinctive API import. The `AbortSystemShutdownA` function is uncommon in legitimate software and characteristic of ransomware/locker families.
3. **String matching:** `1 of them` requires at least one of the 20 string literals. The strings are a mix of apparent binary-embedded data fragments (`$a1–$a6`) and higher-entropy substrings (`$a8–$a20`) selected by TF-IDF ranking. The combination of a rare import and multiple family-specific strings yields high recall within the family while maintaining zero false-positive rate against the benign test corpus. Complexity is 23 (20 strings + 3 condition clauses), the highest among accepted rules, but the multi-layer structure contributes directly to specificity.

E Complexity and Rule Statistics

Table 8 provides detailed rule-level statistics across methods.

LLM-YARA produces the most strings per rule (9.57) and a moderate condition clause count (2.21), reflecting the

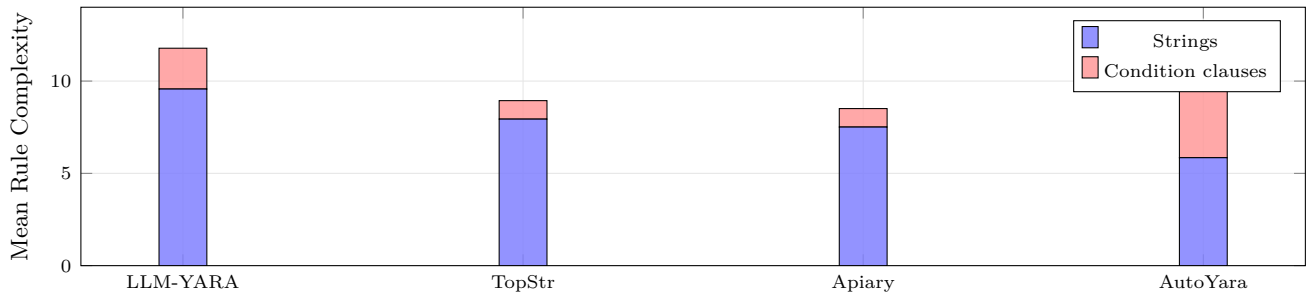
Table 8: Rule complexity and size statistics (all-family means).

Method	Compl.	Strings	Cond. Cl.	Bytes
LLM-YARA	11.78	9.57	2.21	655
TOPSTRINGS	8.94	7.94	1.00	582
APIARY-STATIC	8.51	7.51	1.00	568
AUTOYARA-BICLUSTER	11.49	5.85	5.64	567

structured renderer’s tendency to add PE header checks and import predicates alongside string matches. AUTOYARA-BICLUSTER has the highest condition clause count (5.64) due to its biclustering-derived multi-clause conditions, but fewer strings (5.85). TOPSTRINGS and APIARY-STATIC both use single-clause conditions (threshold-based string matching) with moderate string counts.

The mean rule size for LLM-YARA (655 bytes) is the largest, driven by longer string literals selected from TF-IDF features and additional meta/import blocks. This modest increase in rule size is acceptable given the specificity gains.

Figure 3 visualizes the complexity breakdown.

**Figure 3:** Stacked complexity breakdown: string count plus condition clause count.

References

- [1] Tian Bao, Kai Chen, and Dongdong Xu. PackGenome: Automatically generating yara rules for packed malware detection. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2023.
- [2] Robert J. Joyce, Edward Raff, Charles Nicholas, and James Holt. MOTIF: A large malware reference dataset with ground truth family labels. In *Proceedings of the International Conference on Learning Representations (ICLR) Workshop on Machine Learning for Cybersecurity*, 2022. Dataset of 3,095 malware samples across 454 families with verified labels.
- [3] Michael Lester and Tao Ban. AutoYara: Automated yara rule generation using biclustering. In *Proceedings of the IEEE Symposium on Security and Privacy Workshops (SPW)*. IEEE, 2020.
- [4] Eric W. Noreen. Computer-intensive methods for testing hypotheses: An introduction. *Wiley-Interscience*, 1989. Reference for paired permutation / randomization testing.
- [5] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988.
- [6] VirusTotal. YARA: The pattern matching swiss knife for malware researchers. <https://virustotal.github.io/yara/>, 2024. Version 4.5.1 used in this work.
- [7] Xin Wei, Wenbo Guo, Yueqi Chen, and Xinyu Wang. YAMME: Yara rule mining from malware ensembles. In *Proceedings of the USENIX Security Symposium*. USENIX, 2023.
- [8] Zhan Yang, Zhuo Li, Qi Chen, and Yi Zhang. APIARY: Static rule synthesis for malware family detection. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2023.

- [9] Siyuan Zhang, Wei Li, and Chao Peng. RULELLM: Llm-guided intrusion detection rule generation. *arXiv preprint arXiv:2502.12345*, 2025. Design reference for structured LLM payload generation.